System Design Answer Scripts (SWE1 Level)

Rohit Jangir

1. Design a Rate Limiter

Answer: A rate limiter prevents clients from exceeding a threshold number of requests. A simple design uses a token bucket stored in Redis. Each user has a bucket that refills at a fixed rate. For each request, we decrement the bucket; if empty, we reject the request. Redis ensures distributed consistency and atomicity using INCR and EXPIRE operations.

2. Design a Caching Layer

Answer: Caching reduces database load by storing frequently accessed data in faster memory like Redis. The application checks the cache before hitting the database. TTL is used to prevent stale data. Eviction policies like LRU, LFU, or FIFO maintain memory effectively. Cache hit rate, eviction count, and stale reads are key metrics.

3. Design URL Shortener

Answer: A short URL is generated using a base-62 encoded ID or a hash of the long URL. The system stores mappings in a database. Redirection is a simple lookup. To scale, we can shard IDs, use a distributed key generator, and put a cache in front for faster lookups. Read-to-write ratio is high, so caching helps significantly.

4. Design a Logging System

Answer: Client applications send structured logs to a collector service. Logs are buffered and pushed into a message queue such as Kafka or RabbitMQ. Consumers store them into a search system like Elasticsearch. Dashboards and alerts consume the indexed logs. Key requirements include high throughput, durability, and horizontal scaling.

5. Design Metrics Ingestion (like New Relic)

Answer: Clients submit time-series metrics. The collector normalizes and batches them into a message queue. A worker service stores them in a time-series optimized database like Cassandra or ClickHouse. We expose APIs for querying metrics, and support p95, p99

calculations through aggregations. Challenges include compression, retention, and high write throughput.

6. Design a Job Scheduler

Answer: A scheduler stores jobs with their execution time. A min-heap sorted by timestamp is used to determine the next job to run. Workers poll and execute due jobs. For distributed mode, use a leader-election mechanism (e.g., via Zookeeper or Redis locks) to avoid duplicated execution.

7. REST API Design Principles

Answer: REST models resources using nouns and communicates state using standard HTTP methods. Endpoints are stateless, and request/response formats are typically JSON. Clear versioning, idempotent methods, pagination, rate limiting, and proper status codes improve reliability and scalability.